

Introduction to Data Structures

Data Structures are one of the fundamental concepts in Computer Science and programming. Before diving into Data Structures, let's first understand:

What is Data:

Data refers to the information that can be stored and processed by a computer. It could be anything like numbers, text, images, videos etc.

- In essence, data is the raw material that computers work with to provide meaningful outputs.

What is Data Structure:

A data structure is a systematic way of organizing, storing and managing data to enable efficient access and modification.

It provides a framework for arranging data in a computer's memory or storage, allowing for operations such as retrieval, insertion, deletion and updating to be performed effectively.

Data structures are foundational in computer science and are critical for solving complex computational problems, optimizing performance and ensuring efficient use of resources in software systems.

Why are Data Structures important?

Data structures are the backbone of efficient programming because:

1. They optimize performance: Helps us to process and retrieve data faster.

2. They save resources: efficient use of memory and computational power.
3. They are essential for problem solving: used in databases, web development, AI and more.

→ Objectives of Data Structures:

Data structures satisfy two complementary objectives:

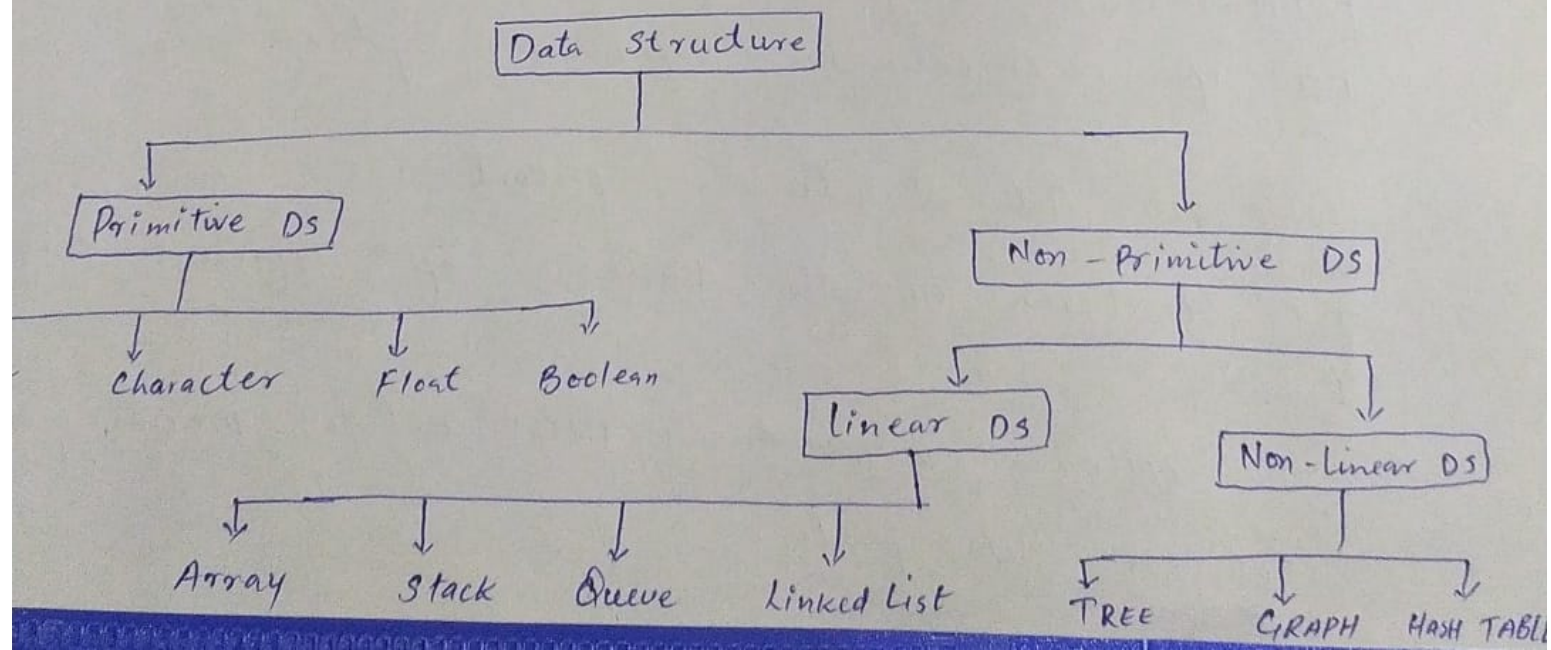
1. Correctness: They must work accurately for all types of inputs within their intended purpose. The primary goal of a data structure is to solve specific problems correctly.
2. Efficiency: They should handle data quickly while using minimal computer resources like memory. In real-world applications, the efficiency of a data structure is crucial for the success of a system or process.

Classification of DS:

A data structure is a way to organize and connect related pieces of data.

We can classify data structures into two categories:

1. Primitive Data Structure
- Non-Primitive Data Structure.



Primitive Data Structures

1. Primitive Data Structures are the DS consisting of the numbers and the characters that come inbuilt into programs.
2. These Data Structures can be manipulated or operated directly by machine-level instructions.
3. Basic data types like integer, character, float, Boolean come under the primitive Data structure.
4. These data types are also called simple data types as they contain characters that cannot be divided further.

Non-primitive Data Structures :

1. Non-primitive DS are those data structures derived from primitive data structures.
2. These data structures can't be manipulated or operated directly by machine-level instructions.
3. The focus of these data structures is on forming a set of data elements that is either homogeneous (same data type) or heterogeneous (different data types).
4. Based on the structure and arrangement of data we can divide these data structures into two sub-categories:

(a) Linear Data Structure

(b) Non-linear Data Structure.

Difference b/w Primitive and non-primitive DS:

Primitive DS	Non-Primitive DS
1. Primitive Data Structure is a kind of DS that allows you to store the data of only one type.	1. Non-primitive DS is a type of data structure that can store the data of more than one type.
2. Examples of primitive DS are: integer, character, float, boolean.	2. Examples of non-primitive DS are: Array, linked list, stack.
3. Primitive DS always contain some value i.e. these data structures do not allow you to store NULL value.	3. You can store a NULL value in non-primitive data structures.
4. The size of primitive DS is dependent on the type of primitive DS.	4. The size of non-primitive DS is not fixed.

→ what is Linear Data Structure

A linear data structure is a type of data structure that stores the data linearly or sequentially. In the linear data structure, data is arranged in such a way that one element is adjacent to its previous and the next element. It includes the data at a single level such that we can traverse all the data into a single run.

- The implementation of the linear data structure is always easy as it stores the data linearly.
- ↳ However, it is not necessarily true in the case of memory, as the arrangement may not be sequential.

Based on memory allocation, the linear DS are further classified into two types:

1. Static Data Structure: The DS having a fixed size are known as static data structures. The memory of these data structures is allocated at compile time, and their size cannot be changed by the user after being compiled. However, the data stored in them can be changed.

Example: ARRAY.

2. Dynamic Data Structure: The data structure having the dynamic size are k/a dynamic DS. The memory of these data structures is allocated at run-time, and their size varies during the run time of the code. Moreover, the user can change the size as well as the data elements stored in these DS.

Examples: Linked List, Queue, and Stack.

TYPES OF LINEAR DATA STRUCTURES:

These include arrays, linked list, stack and Queue.

→ What is non-linear Data Structure

Non-linear data structures are data structures where the data elements are not arranged in sequential order.

- Since the DS is non-linear, it does not involve a single level. Therefore, a user can't traverse all of its elements in a single run.
- The non-linear DS is not very easy to implement as compared to linear DS.
- There exists a hierarchical relationship b/w the individual data items.
- The utilization of computer memory is more efficient in non-linear DS.

→ Types:

1. Trees
2. Graphs.

→ Difference b/w linear and non-linear DS:

Linear DS	Non-linear DS
1. In a linear DS, the data elements connect to each other sequentially.	1. In a non-linear DS, the data elements connect to each other hierarchically.
2. Linear DS are easy to implement.	2. These are difficult to implement and understand as compared to linear DS.
3. A user can find all data elements at a single level.	3. In non-linear DS, data elements are stored at multiple levels.
4. In linear DS, we can traverse all data elements in single run.	4. It's not easy to traverse non-linear DS. The users need to multiple runs to traverse them completely.
5. They can't utilize memory efficiently.	5. memory utilization is efficient.
Time complexity is directly proportional to its size size.	6. Time complexity often remains same with an increase in its i/p size.
Ex: Linked list, stack, Queue, Array.	7. Ex: GRAPH, Tree, etc.

Basic operations of Data Structures:

Data Structures support several essential operations that enable the efficient management and manipulation of data.

Below are the key operations that are performed on DS:

1. Traversal: is the process of visiting each element in a data structure exactly once to access or process it. This operation is critical for tasks like displaying data or performing computations.

- Depending on the type of DS, traversal can follow different patterns such as linear traversal in arrays or linked lists and depth-first or breadth-first traversals in trees or graphs.

2. Insertion: is the process of adding new elements to a data structure. The operation helps grow the dataset by adding additional information.

- Insertion location can vary based on DS, such as insertion at end in an array, or at the head of linked list.

3. Deletion: it means to remove or delete a specific data element from a data structure.

- The deletion process often requires updating references or shifting elements, as in array, linked list or binary trees.

4. Searching: is the operation of finding the location of a particular data element in a data structure that meets certain criteria. Such a data element may or may not be present in a given set of data elements.

5. Sorting: is the process of arranging elements in a specific order, such as ascending or descending order.

This makes data more accessible and understandable.

- Sorting algorithms are used like selection sort, bubble sort, insertion sort etc. depending on the complexity and size of dataset.

6. Merge: means to combine data elements of two sorted lists in order to form a single list of sorted data elements.

7. Splitting: involves dividing a large data structure into smaller parts to improve processing efficiency or for logical separation.

- splitting is often done based on specific conditions, such as splitting a tree into subtree.

8. Update: The update operation allows us to update or modify the data in a data structure.

9. Selection: means selecting a particular data from the available data. we can select any particular data by specifying conditions during traversal.

10. Creation: is the operation of initializing a new data structure, including allocating memory and defining its layout.

- In programming, functions like `malloc()` in C or Constructors in object-oriented programming are used for this operation.

ARRAYS

An array is a linear data structure that stores the collection of elements of the same type in contiguous memory locations.

Each element in an array can be identified and accessed using an index, which is typically a numerical value starting from 0 to $n-1$, where n is the size of array.

Memory Representation

In an array, all the elements are stored in contiguous memory locations. So if we initialize an array, the elements will be allocated sequentially in memory.

100	104	108	112	116	120	124	128	132
11	12	74	2	9	7	6	14	19
0	1	2	3	4	5	6	7	8

→
index.

Declaration of Array:

Syntax:

`int arr[5];` → `dataType arrayName[size]`

`float arr[10];`

Initialization:

`int arr[5] = { 1, 2, 7, 40, 6 };`



The name of the array represents the base address or the address of the first element in the main memory.

Traversal of an array:

```
#include <stdio.h>
```

```
int main () {
```

```
    int arr[5] = { 1, 2, 7, 9, 6 };
```

```
    int i;
```

```
    printf("Elements of array are : \n");
```

```
    for(i=0; i<5; i++) {
```

```
        printf("%d", arr[i]);
```

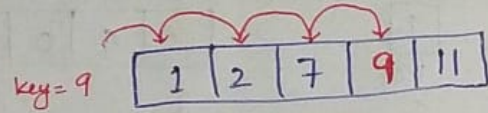
```
    }
```

```
    return 0;
```

```
}
```


LINEAR SEARCH ALGORITHM

Linear Search AKA Sequential Search is a method for searching an element in a collection of elements. In linear search, each element of the collection is visited one by one in a sequential fashion to find the desired element.



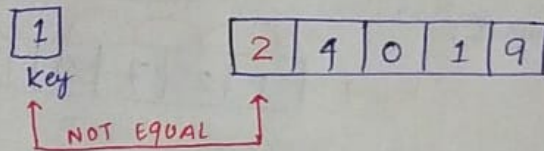
STEPS:

1. Start: Begin at the first element of the collection of elements.
2. Compare: Compare the current element with the desired element.
3. Found: If the current element is equal to the desired element, return True or index of current element.
4. move: Otherwise move to next element in the collection.
5. Repeat: Repeat step 2-4 until we have reached the end of collection.
6. NOT Found: If the end of collection is reached without finding the desired element, return that desired element is not in the array.

→ Steps to find/Search for an element $key=1$ in an array.

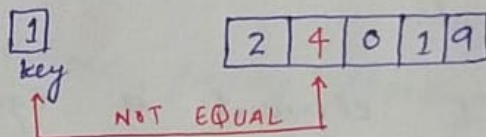
2	4	0	1	9
0	1	2	3	4

1. Start from first element (index 0) and compare key with each element.



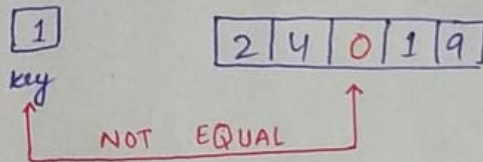
Since key and current element are not equal, iterator moves to next element.

2. Compare key with next element $arr[1]$.

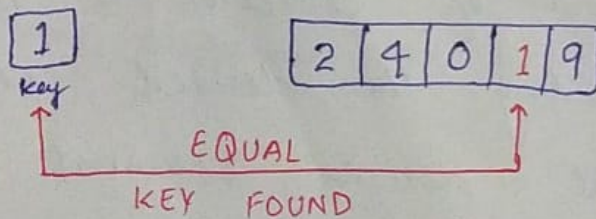


move iterator to next element.

3. Compare key with next element $arr[2]$



4. Compare key with next element $arr[3]$.



Now comparing key with $arr[3]$, the value matches. So the Linear Search Algorithm will yield Successful message.

Implementation :

```
#include <stdio.h>
```

```
int Search (int arr[], int size, int key) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        if (arr[i] == key) {
```

```
            return 1; // or return i (index will be returned).
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```
int main() {
```

```
    int num[10] = {1, 2, 5, 3, -3, -11, 17, 100, 6, -7};
```

```
    int size = 10;
```

```
    int key;
```

```
    printf("Enter the key you want to find : ");
```

```
    scanf("%d", &key);
```

```
    // Function call
```

```
    int found = Search(num, size, key);
```

```
    if (found == 1) {
```

```
        printf("key is present");
```

```
    } else {
```

```
        printf("key is absent");
```

```
    }
```

```
    return 0;
```

```
}
```


Linear Search also called the sequential search. It is the simplest searching algorithm. In linear search, we simply traverse the collection of elements completely and match each element of the collection with the item whose location is to be found. If the match is found then the location of item is returned otherwise algorithm returns 0 or -1.

LS Time and Space Complexity:

Te:

- Best case: In the best case, the key might be present at the first index. So, the best case complexity is $O(1)$.
- Worst case: In the worst case, the key might be present at the last index. So, worst case complexity is $O(N)$ where N is the size of the list.
- Average case: $O(N)$.

Sc: $O(1)$ as except for the variable to iterate through the list, no other variable is used.

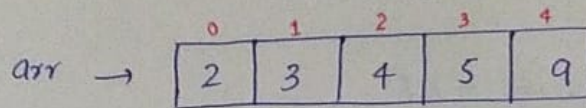
Advantages:

- LS can be used irrespective of whether the array is sorted or not.
- Does not require any additional memory.
- It is well suited algorithm for small data-sets.

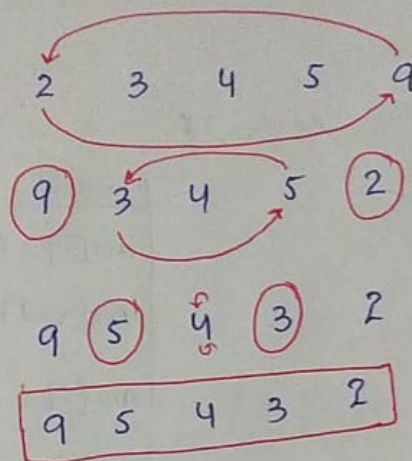
Disadvantages:

- It has time complexity of $O(N)$, which in turn makes it slow for large dataset.
- Not suitable for large arrays.

Q: Reverse an Array?



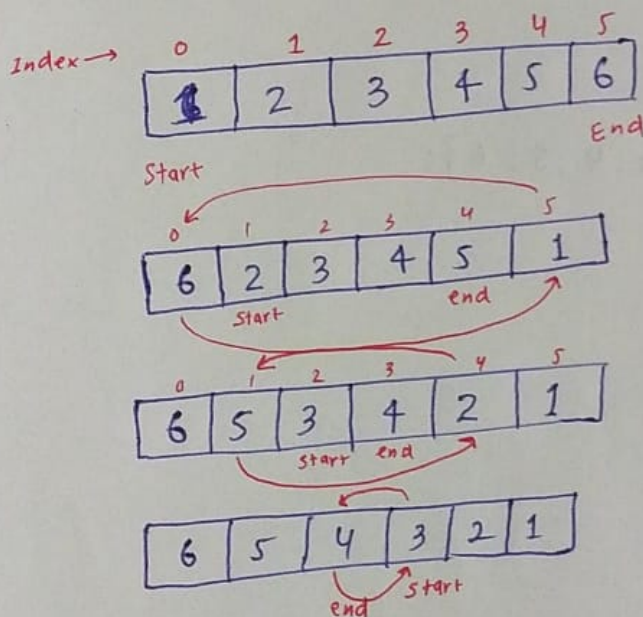
Logic:



we are swapping the first element of array i.e. $arr[0]$ with the last element i.e. $arr[n-1]$ and then incrementing first index to next & decrementing last index to $n-1$.

There are two cases:

odd array $\rightarrow$ Even array.



when $start > end$
STOP.

Algo: 1. Swap (start, end);

2. start ++ , end --

Code:

```
#include <stdio.h>

void reverse (int arr[], int n) {
    int end = n-1;
    for (int i = 0 ; i <= end; i++, end--) {
        int temp = arr[i];
        arr[i] = arr[end];
        arr[end] = temp;
    }
}
```

without Temp:

```
arr[i] = arr[i] + arr[end];
arr[end] = arr[i] - arr[end];
arr[i] = arr[i] - arr[end];
```

```
void PrintArray (int arr[], int n) {
    for (int i = 0; i < n ; i++) {
        printf("%d", arr[i]);
    }
}
```

```
int main () {
```

```
    int num[6] = { 1, 2, 3, 4, 5, 6 };
```

```
    reverse (num, 6);
```

```
    PrintArray (num, 6);
```

```
    return 0;
```

```
}
```


OPERATIONS ON ARRAY

Insertion: refers to the process of inserting an element in a given array.

There are two cases.

CASE 1: Insert an element in an array while maintaining the order of elements.

Ex:

Original Array:

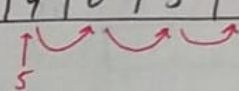
Arr[100] →

0	1	2	3	4	...	99
7	3	9	0	5		

→ Now, Insert 5 at index 2.

For that we have to make space for 5 at index 2 by shifting the elements to right after that index.

0	1	2	3	4	...	99
7	3	9	0	5		



5 → index 2.

Arr[100] →

0	1	2	3	4	5	...	99
7	3	5	9	0	5		

BEST CASE

Best case for inserting an element in an array would be at the last index of array where we have not to move any other element.

Original Array:

0	1	2	3	...	99
1	2	4	9		

insert → 7

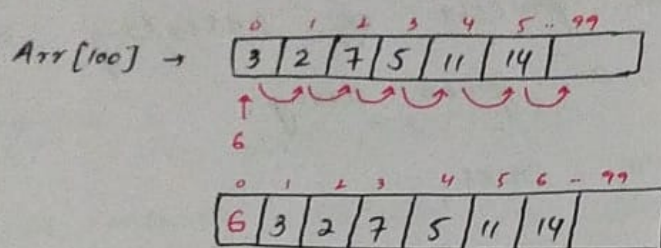
0	1	2	3	4	...	99
1	2	4	9	7		

TC: $O(1)$

WORST CASE:

when we have to insert element at 0 index.

Ex:

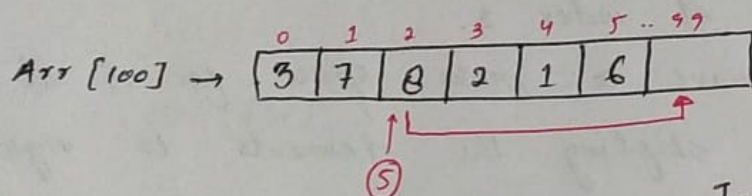


TC → $O(n)$

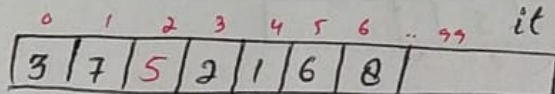
CASE 2: Insert element in an array without maintaining the order of elements.

→ insert 5 → index 2.

Ex:



Just take that element at given index and put it at last.



Implementation of Insertion

```
#include <stdio.h>

int indInsertion (int arr[], int size, int element, int index,
                  int capacity) {

    if (size >= capacity) {
        return -1;
    }
    } → edge case

    for (int i = size - 1; i >= index; i--) {
        arr[i+1] = arr[i];
    }
    arr[index] = element;
    return 1;
}

void printArray (int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}

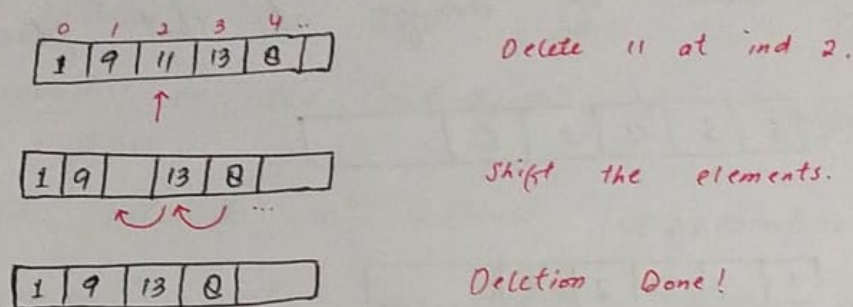
int main () {
    int arr[100] = { 1, 2, 4, 6, 7 };
    int size = 5;
    int index = 2;
    int element = 45;
    int capacity = 100;

    indInsertion (arr, size, element, index, capacity);
    size += 1;
    printArray (arr, size);

    return 0;
}
```

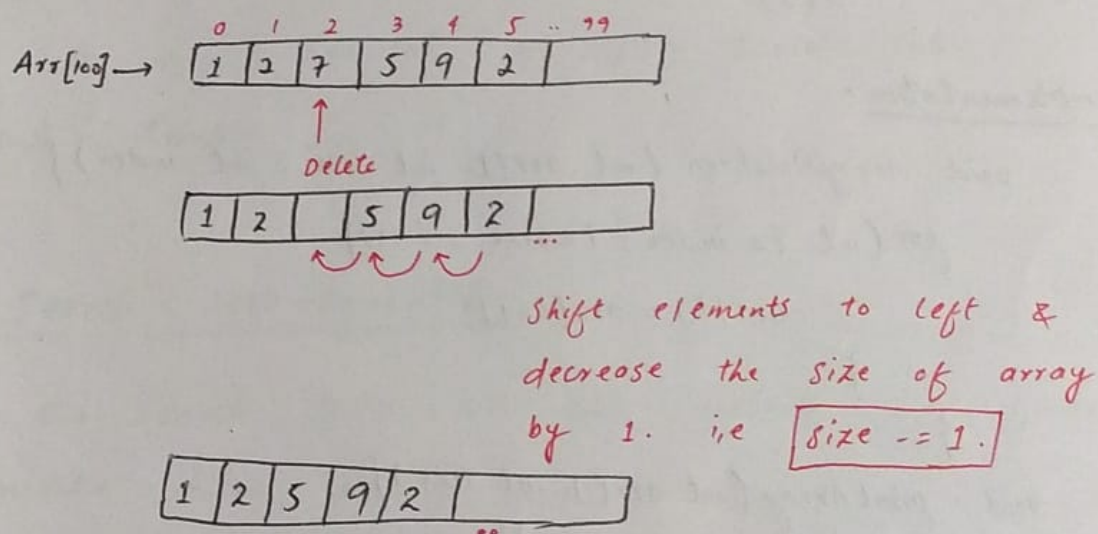
Deletion :

An element at specified position can be deleted by creating a void which needs to be fixed by shifting all the elements to the left.



There are two cases of deletion as well :

CASE 1 : Deleting the element while maintaining the order of Array.



TC :

- Best Case $\rightarrow$ Deleting the last element $= O(1)$
- Worst Case $\rightarrow$ Deleting the first element and Shifting all elements to left $\rightarrow O(n)$

CASE 2: Deleting an element and not maintaining the order of Array. i.e.

Deleting the element at given index and filling that void space by putting the last element of array at that place.

Arr[100] →

0	1	2	3	4	...	99
1	5	11	2	6		

Delete 5 → at ind 1.

↑
Delete

1		11	2	6		
---	--	----	---	---	--	--

↖

Size -= 1

1	6	11	2			
---	---	----	---	--	--	--

↖

Time Complexity:

$O(1)$

Implementation:

```
void ArrayDeletion (int arr[], int size ; int index) {
```

```
    for (int i = index ; i < size ; i++) {
```

```
        arr[i] = arr[i+1];
```

```
    }
```

```
}
```

```
void printArray (int arr[], int size) {
```

```
    for (int i = 0 ; i < size ; i++) {
```

```
        printf ("%d " arr[i]);
```

```
    }
```

```
int main () {
```

```
    int arr[100] = { 1, 2, 5, 7, 56, 9, 13 };
```

```
    int size = 7;
```

```
    int index = 0;
```

```
    ArrayDeletion (arr, size, index);
```

```
    size -= 1;
```

```
    printArray (arr, size);
```

```
    return 0;
```

```
}
```

BINARY SEARCH ALGORITHM

Binary Search is a search algorithm used to find the position of a target value within a sorted array. It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty.

→ The search interval is halved by comparing the target element with the middle value of the search space.

Conditions to Apply Binary Search :

- The DS must be sorted.
- Access to any element of DS should take constant time.

Binary Search step-by-step

1. Divide the search space into two halves by finding the middle index "mid".
2. Compare the middle element of the search space with the key.
3. If the key is found at middle element, the process is terminated.
4. If the key is not found at middle element, choose which half will be used as the next search space :

- If the key is smaller than the middle element, then the left side is used for next search.
 - If the key is larger than the middle element, then the right side is used for next search.
5. The process is continued until the key is found, or the total search space is exhausted.

WORKING :

→ Consider an array :

$$\text{arr}[10] = \{2, 5, 8, 12, 16, 23, 38, 56, 72, 91\}$$

Initially :

	0	1	2	3	4	5	6	7	8	9
arr[10] =	2	5	8	12	16	23	38	56	72	91

key = 23

STEP 1:

	0	1	2	3	4	key	6	7	8	9
arr[10] =	2	5	8	12	16	23	38	56	72	91
	start=0				mid=4		end=Size-1 i.e 9.			

$$\text{mid} = \frac{(\text{start} + \text{end})}{2} = \frac{0+9}{2} = \frac{9}{2} = 4$$

STEP 2:

$$\rightarrow 16 \neq 23$$

$$\rightarrow 23 > 16$$

Search space moves to right

STEP 3:

	5	6	7	8	9
	23	38	56	72	91
	start=5		mid=7	end=9	

$$\text{mid} = \frac{5+9}{2} = \frac{14}{2} = 7$$

56 $\neq$ 23 , 23 < 56 , so search space moves to left.

Key = 23.

5	6
23	38

 start = 5 end = 6
 mid = 5

$$23 == 23$$

↓
TRUE

↳ Return (5)

→ ans = 5

Code Implementation :

```
#include <stdio.h>
```

```
int binarySearch (int arr[], int size, int key) {
```

```
    int start = 0;
```

```
    int end = size - 1
```

```
    int mid = start + (end - start) / 2 ;
```

```
    while (start <= end) {
```

```
        if (key == arr[mid]) {
```

```
            return mid ;
```

```
        }
```

```
        if (key > arr[mid]) {
```

```
            start = mid + 1;
```

```
        } else {
```

```
            end = mid - 1;
```

```
        }
```

```
        mid = start + (end - start) / 2 ;
```

```
    }
```

```
    return -1;
```

```
}
```

```
int main() {
```

```
    int num[6] = { 1, 3, 5, 6, 8, 11 };
```

```
    int key = 8;
```

```
    int index = binarySearch (num, 6, key);
```

```
    printf ("Index of %d is %d", key, index);
```

```
    return 0; }
```


2D Arrays

A 2D array is a collection of elements organized into rows and columns, forming a grid like structure.

→ It is essentially an array of arrays, where each element can be uniquely identified using two indices one for row and one for column.

→ In memory, a 2D array is stored in contiguous memory locations.

↳ In C, it is stored in row-major-order (all elements of a row are stored before moving to the next row).

↳ Address Calculation formula:

$$C \times i + j$$

$i \rightarrow$ rows.

$j \rightarrow$ columns.

$C \rightarrow$ Total no. of columns.

Ex: 1st row, 0th col.

Total 3 cols

$$3 \times 1 + 0 = \boxed{3}$$

Syntax:

`datatype arrayName[rowSize][columnSize];`

Declaration:

`int arr[3][4];`

Initialization: 2D array can be initialized in several ways:

① `int matrix[2][3] = { {1, 2, 3},
 {4, 5, 6} };`

② Flattened initialization:

```
int matrix[2][3] = { 1, 2, 3, 4, 5, 6 };
```

③ After declaration assigning values to each element individually:

```
matrix[0][0] = 1;
```

```
matrix[0][1] = 2;
```

Example:

```
#include <stdio.h>
```

```
int main () {
```

```
    int matrix[2][3] = { { 1, 2, 3 },  
                          { 4, 5, 6 } };
```

```
    // Print
```

```
    printf("matrix : \n");
```

```
    for(int i=0; i<2; i++) {
```

```
        for(int j=0; j<3; j++) {
```

```
            printf("%d", matrix[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```


2D - ARRAYS

Two-dimensional arrays offer a structured way to store and manipulate data in grid-like format, ideal for applications requiring tabular data representation such as matrices and game boards.

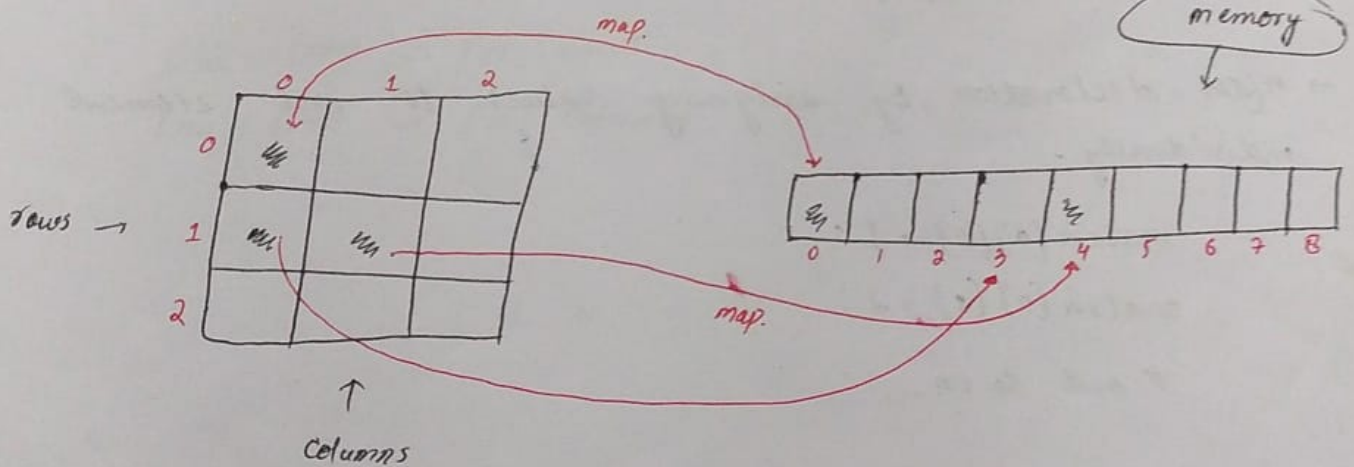
→ 2D-Arrays allows to store data in rows and columns.

Syntax :

`dataType arrayName[rowSize][columnSize];`

Memory : In memory 2D arrays are stored in linear array.

→ 3 rows × 3 columns.



mathematical formula :

$$C \times i + j$$

$i \rightarrow$ rows

$j \rightarrow$ columns.

$C \rightarrow$ Col. (Total no. of Col).

1st row, 0th column

$$= 3 \times 1 + 0$$

$$= 3$$

Iterate over a 2D Array.

```
#include <stdio.h>
```

```
int main() {
```

```
    int i=0, j=0;
```

```
    int grid[2][3] = { {10, 20, 30}, {5, 10, 15} };
```

```
    // looping through 2D array.
```

```
    for(i=0; i<2; i++) {
```

```
        for(j=0; j<3; j++) {
```

```
            printf("grid[%d][%d] = %d \n", i, j, grid[i][j]);
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Outer loop → for rows

Inner loop → for columns.

Filling a 2D Array and Displaying it :

```
int main() {  
    int matrix[2][3];  
    int i, j;  
  
    for(i=0; i<2; i++){  
        for(j=0; j<3; j++){  
  
            printf("Enter the value for matrix[%d][%d]: ", i, j);  
            scanf("%d", &matrix[i][j]);  
        }  
    }  
  
    printf("\n Displaying the matrix : \n");  
    for(i=0; i<2; i++){  
        for(j=0; j<3; j++){  
            printf("%d\t", matrix[i][j]);  
        }  
        printf("\n");  
    }  
}
```

DYNAMIC MEMORY ALLOCATION IN 2D ARRAY

To dynamically allocate memory for a 2D array in C, we typically use a pointer to a Pointer (`dataType ** arrayName`) along with the `malloc` function, which allocates the specified amount of memory and returns a pointer to it.

Syntax:

```
dataType ** arrayName;  
arrayName = (dataType **) malloc (numberOfRows * sizeof (dataType *));  
for (int i=0; i < numberOfRows; i++) {  
    array[i] = (dataType *) malloc (numberOfColumns * sizeof (dataType));  
}
```

Freeing the memory:

To properly free the memory allocated for a 2D array, we should first free each row (i.e. each `*int` pointer within the array of pointers) and then free the pointer to the array of pointers itself (`int **`). This ensures that all allocated memory is returned to the system.

```
for (int i=0; i < numberOfRows; i++) {  
    free (arrayName[i]);  
}
```

```
free (arrayName);
```


2D Array operations

Addition of 2 matrices

```
#include <stdio.h>

void displaymatrix (int matrix [2][3]) {
    for (int i=0; i<2; i++) {
        for (int j=0; j<3; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
}

int main () {
    int mat1 [2][3] = { {1, 2, 3}, {4, 5, 6} };
    int mat2 [2][3] = { {7, 8, 9}, {10, 11, 12} };
    int Sum [2][3];

    for (int i=0; i<2; i++) {
        for (int j=0; j<3; j++) {
            Sum[i][j] = mat1[i][j] + mat2[i][j];
        }
    }

    printf("matrix 1: \n");
    displaymatrix (mat1);

    printf("matrix 2: \n");
    displaymatrix (mat2);

    printf("Addition of two matrices: \n");
    displaymatrix (Sum);

    return 0;
}
```

Same code for Subtraction
Just change sign in this line.

② Transpose:

Rows $\leftrightarrow$ Columns

```
#include <stdio.h>
#define Rows 3
#define COLUMNS 3
void displaymatrix (int matrix[][COLUMNS]) {
    for(int i=0; i<Rows; i++){
        for(int j=0; j<COLUMNS; j++){
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
}

int main () {
    int mat[Rows][COLUMNS] = { {1, 2, 3},
                                   {4, 5, 6},
                                   {7, 8, 9} };

    int trans[COLUMNS][Rows];
    for(int i=0; i<Rows; i++){
        for(int j=0; j<COLUMNS; j++){
            trans[j][i] = mat[i][j];
        }
    }
    printf("matrix : \n");
    displaymatrix(mat);

    printf("Transpose is : \n");
    displaymatrix(trans);

    return 0;
}
```


Multiplication

```
#include <stdio.h>
#define Rows1 2
#define Columns1 3
#define Rows2 3
#define Columns2 2

void displaymatrix (int matrix[][Columns2], int rows, int columns) {
    for (int i=0; i < rows; i++) {
        for (int j=0; j < columns; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
}

int main () {
    int mat1[Rows1][Columns1] = { {1, 2, 3}, {4, 5, 6} };
    int mat2[Rows2][Columns2] = { {7, 8}, {9, 10}, {11, 12} };
    int Product[Rows1][Columns2] = {0};

    if (Columns1 != Rows2) {
        printf("Matrix multiplication not possible\n");
        return 1;
    }

    // multiplication
    for (int i=0; i < Rows1; i++) {
        for (int j=0; j < Columns2; j++) {
            for (int k=0; k < Columns1; k++) {
                Product[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }

    printf("Result matrix :\n");
    displaymatrix (Product, Rows1, Columns2);
    return 0;
}
```

SPARSE MATRIX :

matrix: A matrix can be defined as a two dimensional array having m rows and n columns. A matrix with m rows and n columns is called $m \times n$ matrix.

SPARSE MATRIX :

Sparse matrices are those that have the majority of their elements equal to Zero.

In other words, the sparse matrix can be defined as the matrix that has a greater number of Zero elements than non-Zero elements.

Why sparse matrix .

There are following benefits of using the Sparse matrix:

Storage: We know that a sparse matrix contains lesser non-Zero elements, so less memory can be used to store elements. It evaluates only the non-Zero elements.

Computing time: In the case of searching in sparse matrix, we need to traverse only non-Zero elements rather than traversing all the sparse matrix elements. It saves computing time by logically designing a data structure traversing non-Zero elements.

Example:

0	0	3	0	4
0	0	5	7	0
0	0	0	0	0
0	2	6	0	0

Representing a sparse matrix by a 2D array leads to wastage of lots of memory as zeros in the matrix are of no use in most of cases. So, instead of storing zeros with non-zero elements, we only store non-zero elements. This means storing non-zero elements with triples - (Row, column, value).

Representation of sparse matrix using Array:

2D array is used to represent a sparse matrix in which there are 3 rows named as:

- Row: index of row, where non-zero element is located.
- Column: index of column, where non-zero element is located.
- Value: value of non-zero element located at index (row, column).

$$\begin{bmatrix} 0 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 6 & 0 & 0 \end{bmatrix}$$

→

Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

Time Complexity:

$O(Nm)$, where N is the number of rows in sparse matrix, and m is the number of columns in sparse matrix.

$O(Nm)$.

RECURSION

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called recursive function.

- A recursive function solves a particular problem by calling itself with different parameters to solve smaller subproblems of the original problem. The called function may further call itself and this process may continue forever. So, for that purpose we write a base case, where the function will stop.

Important Concepts:

◦ Base case: The condition where recursion stops. Without a base case, recursion leads to infinite calls.

◦ Recursive Case: The part of the function where it calls itself with modified arguments to approach the base case.

How are Recursive functions stored in memory:

Recursion uses more memory, because recursive function adds to the stack with each recursive call, and keeps values there until the call is finished. The recursion uses LIFO structure just like the stack. DS.

Example: Factorial of a number:

```
#include <stdio.h>

int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n-1);
}

int main() {
    int n = 5;
    int x = factorial(n);
    printf("Factorial of %d is %d : ", n, x);
    return 0;
}
```

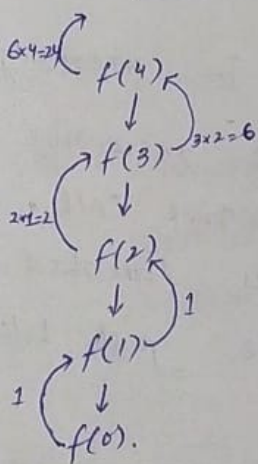
Tc: $O(n)$.
Sc: $O(n)$

→ Recursion Time Complexity:

① Recurrence Relation.

② Tc = Total no. of Recursion calls * work done in each call.

Rec. Tree.



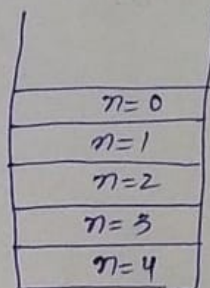
$n \sim n$ calls.

$$Tc = O(n \times k)$$

$$\Rightarrow O(n)$$

Rec:

height of call stack
*
memory in each call.



Call Stack.

Selection Sort

Selection Sort is a comparison based algorithm. It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

Algorithm:

Step 1: Set min to location 0

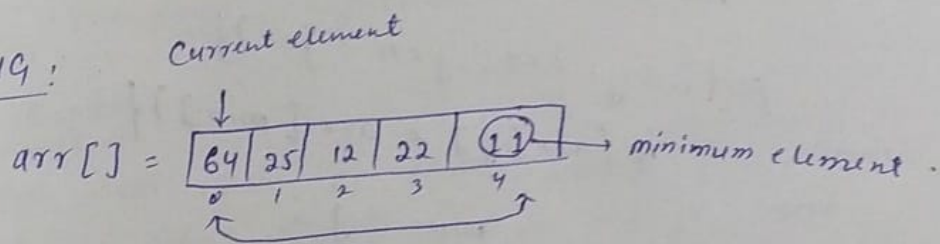
Step 2: Search the minimum element in list.

Step 3: Swap the smallest element with the element at index min.

Step 4: Increment min to next index

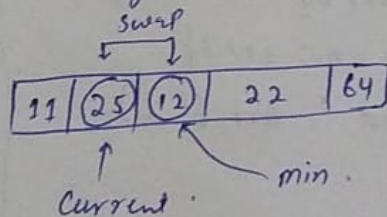
Steps: Continue all elements are sorted.

WORKING:

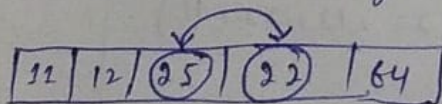


①: Start from first element at index 0, find the smallest element in the rest of array which is unsorted, and swap (11) with current element.

② move to next element at index 1 (25), find smallest in remaining array, and swap (12) with current element.



③ move to element at index 2 (25). find smallest element from unsorted subarray, swap (22) with current element (25)



- ④ move to element at index 3 (25), find min. element from unsorted array, Swap (25) with current element.

11	12	22	25	64
0	1	2	3	4

- ⑤ move to next element at index 4, find min element from unsorted sub array, swap with current element.

11	12	22	25	64
0	1	2	3	4

We got the sorted array.

CODE:

```
#include <stdio.h>
void SelectionSort (int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
```

```
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min]) {
                min = j;
```

```
            }
        }
```

```
        int temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
```

```
    }
}
```

```
void Printarray (int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
int main () {
    int arr[5] = { 64, 25, 12, 22, 11 };
    int size = 5;
```

```
    printf("Original Array: ");
    Printarray (arr, size);
```

```
    SelectionSort (arr, size);
```

```
    printf("Sorted Array: ");
```

```
    Printarray (arr, size);
```

```
    return 0;
```

```
}
```



Complexity Analysis:

TC: $O(n^2)$, as there are two nested loops:

- one loop to select an element of Array one by one = $O(n)$.
- Another loop to compare that element with every other element = $O(n)$.

$$\text{Overall TC} = O(n) * O(n) = O(n^2)$$

SC: $O(1)$

Advantages of selection sort:

- Easy to understand and implement.
- Requires only a constant $O(1)$ extra memory space.
- It requires less no. of swaps compared to many other standard algorithms.

Disadvantages:

- Selection sort has TC $O(n^2)$ makes it slower, compared to algorithms like Quick sort or merge sort.
- Does not preserve relative order of elements.

Bubble Sort

Bubble sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large datasets as its average and worst-case time complexity is quite high.

- We sort the array using multiple passes. After the first pass, the maximum element goes to end (its correct position). Same way, after second pass, the second largest element goes to second last position and so on.
- In every pass, we process only those elements that have already not moved to correct position. After k passes, the largest k elements must have been moved to the last k position.
- In a pass, we consider remaining elements and compare all adjacent and swap if larger element is before a smaller element. If we keep doing this, we get the largest (among the remaining elements) at its correct position.

Working:

① Placing the first largest element at its correct position.

$i=0$

5	6	1	3
---	---	---	---

$i=1$

5	6	1	3
---	---	---	---

$i=2$

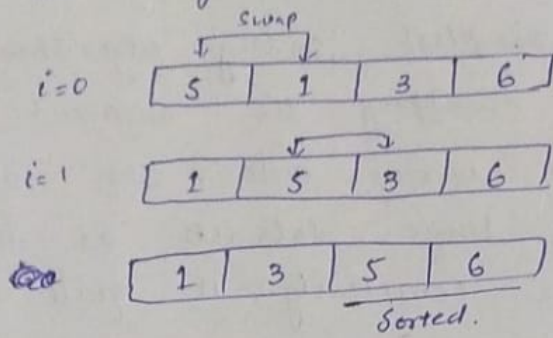
5	1	6	3
---	---	---	---

5	1	3	6
---	---	---	---

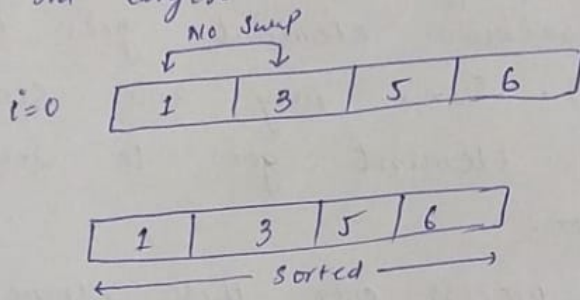
sorted element.

For n elements there will be $n-1$ passes.

② placing 2nd largest element at its correct position



③ placing 3rd largest.



Tc: $O(n^2)$

Sc: $O(1)$

Code:

```
void bubbleSort(int arr[], int size){
```

```
    int i, j, temp;
```

```
    for(i=1; i<size; i++){
```

```
        for(j=0; j<size-i; j++){
```

```
            if(arr[j] > arr[j+1]){
```

```
                temp = arr[j];
```

```
                arr[j] = arr[j+1];
```

```
                arr[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```


Insertion Sort

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.

→ It is like sorting playing cards in your hands.

You split the cards into two groups: the sorted cards and unsorted cards. Then you pick a card from the unsorted group and put it in the right place in the sorted group.

↳ we start with the ~~sorted~~ ^{second} element of the array as the first element in array is assumed to be sorted.

↳ Compare second element with the ~~second~~ ^{first} element and check if the second element is smaller then swap them.

↳ move to the third element and compare it with the first two elements and put at its correct position.

↳ Repeat until entire array is sorted.

Initially

23	1	10	5	2
----	---	----	---	---

 →

23	1	10	5	2
----	---	----	---	---

First Pass

23	1	10	5	2
----	---	----	---	---

 →

1	23	10	5	2
---	----	----	---	---



Second Pass

1	23	10	5	2
---	----	----	---	---

 →

1	10	23	5	2
---	----	----	---	---

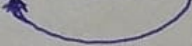


Third Pass

1	10	23	5	2
---	----	----	---	---

 →

1	5	10	23	2
---	---	----	----	---



Fourth Pass

1	5	10	23	2
---	---	----	----	---

 →

1	2	5	10	23
---	---	---	----	----



code: void insertionSort (int arr[], int n) {

for (int i = 1; i < n; i++) {

int curr = arr[i];

int prev = i - 1;

while (prev > 0 && arr[prev] > curr) {

arr[prev + 1] = arr[prev];

prev--;

}

arr[prev + 1] = curr;

}

Time Complexity:

→ Best case = $O(n)$: if list is already sorted.

→ Average case: $O(n^2)$

→ Worst case: $O(n^2)$

Space Complexity:

$O(1)$

Advantages:

- Simple & easy to implement.
- Stable sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space efficient as it is an in-place algorithm.
- Adaptive.

Disadvantages:

- Inefficient for large lists.
- Not as efficient as other sorting algo's (merge sort, QS) for most cases.

STACK

Stack is a linear data structure that follows the LIFO (Last In First Out) principle for inserting and deleting elements from it.

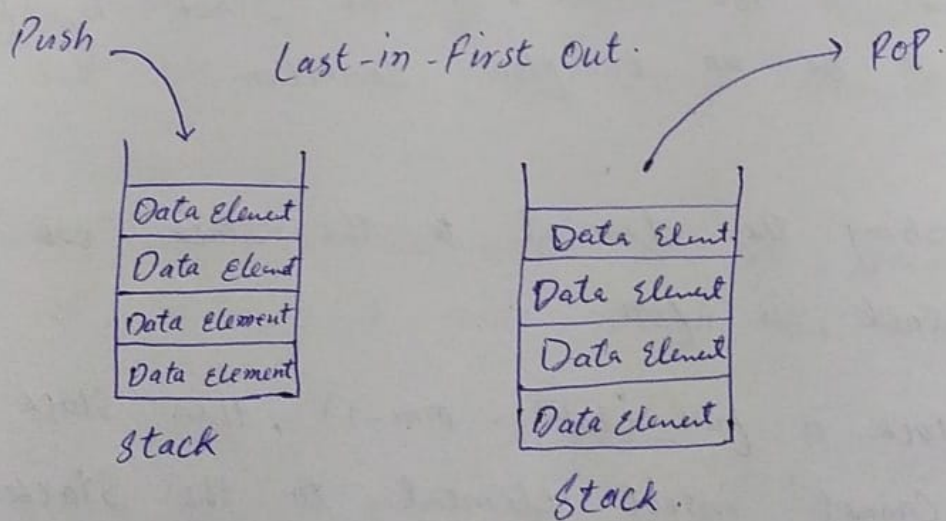
By LIFO, we mean that last element that we insert within the stack is the first element we can remove from it. You can assume a stack data structure as the pile of books on top of another.

In the stack data structure, the position from where we can add or remove elements is called the 'Top' of the stack.

Stack Representation

A stack allows all operations at one end only. At any given time, we can only access the top element of the stack.

Diagram:



A stack can be implemented by means of array, structure, and linked list.

Stack can either be fixed in size or it may have sense of dynamic resizing.

Standard stack operations

① Push(): when we insert an element in the stack then the operation is known as a push. If the stack is full then the overflow condition occurs.

② Pop(): when we delete an element from the stack, the operation is k/a pop. If the stack is empty means that no element exists in the stack, this state is k/a an underflow state.

③ Peek(): it returns the element at the given ~~condition~~ position.

→ Push operation in Stack: $Tc: O(1)$, $Sc: O(1)$

Adds an item to the stack. If the stack is full, then it is said to be an overflow condition.

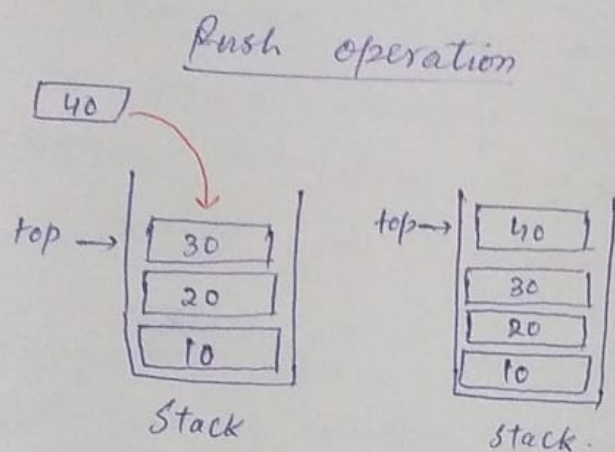
Algorithm:

- Before pushing the element to the stack, we check if the stack is full.

→ If the stack is full ($top == MAX-1$), then stack overflows and we cannot insert element to the stack.

Otherwise, we increment the value of top by 1 and the new value is inserted at top position.

The elements can be pushed into the stack till we reach MAX. Capacity of stack.

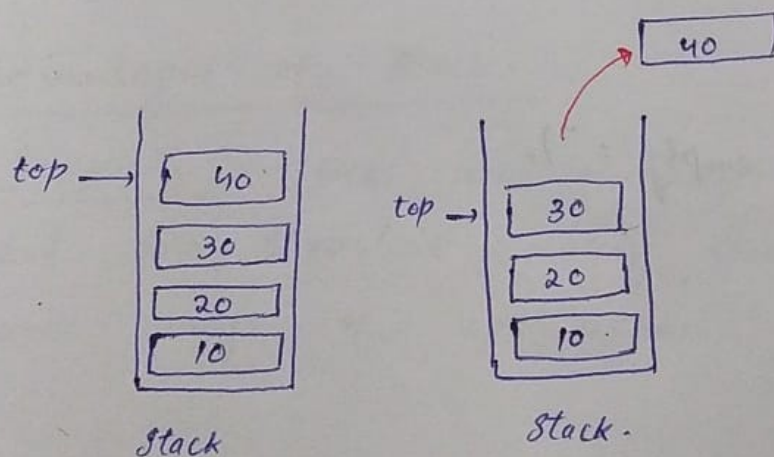


POP operation in Stack : Tc: $O(1)$, Sc: $O(1)$.

Removes an element from the stack. The items are popped in the reverse order in which they are Pushed. If the Stack is empty, then it is said to be an Underflow Condition.

Algorithm for pop operation:

- Before popping the element from the stack, we check if the stack is empty.
- If the stack is empty ($top == -1$), then Stack Underflows and we cannot remove any element from the Stack.
- Otherwise, we store the value at top, decrement the value of top by 1 and return the stored value.



Stack Using Array:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
void Push (int stack [], int *, int );
```

```
int POP (int stack [], int *);
```

```
int main () {
```

```
    int stack[MAX];
```

```
    int top = -1;
```

```
    Push (stack, &top, 10);
```

```
    Push (stack, &top, 20);
```

```
    Printf("\n Popped Element is : %d ", POP(stack, &top));
```

```
    return 0;
```

```
}
```

```
Void Push (int stack [], int *top, int val) {
```

```
    if (*top == MAX - 1) {
```

```
        Printf("\n Stack is Full... ");
```

```
        return;
```

```
    } else {
```

```
        (*top)++;
```

```
        Stack [*top] = val;
```

```
    }  
};
```

```
int POP (int stack [], int *top) {
```

```
    int x;
```

```
    if (*top == -1) {
```

```
        Printf("Stack is empty : ");
```

```
        return -1;
```

```
    } else {
```

```
        x = Stack [*top];
```

```
        Stack [*top] = 0;
```

```
        (*top) --;
```

```
    } return x; }
```


APPLICATIONS OF STACK

- Function Calls: Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing.
- Recursion: Stacks are used to store the local variables and return addresses of recursion function calls, allowing the program to keep track of the current state of the recursion.
- Expression Evaluation: Stacks are used to evaluate expressions in postfix notation. (Reverse Polish Notation).
- Syntax Parsing: Stacks are used to check the validity of syntax in programming languages and other formal languages.
- Memory Management: Stacks are used to allocate and manage memory in some operating systems and programming languages.
- Used to solve popular problems like Next Greater, Previous Greater, Next Smaller, Previous Smaller, Stock Span Problems etc.

Advantages of Stack:

- Simplicity: Stacks are a simple and easy-to-understand data structure, making them suitable for a wide range of applications.

- Efficiency: Push and Pop operations on a stack can be performed in constant time ($O(1)$), providing efficient access to data.
- Limited memory usage: Stacks only need to store the elements that have been pushed onto them, making them memory efficient compared to other data structures.

Disadvantages of Stack:

- Limited access: Elements in the stack can only be accessed from the top, making it difficult to retrieve or modify elements in the middle of the stack.
- Potential for Overflow: If more elements are pushed onto a stack than it can hold, an overflow error will occur, resulting in a loss of data.
- Not suitable for Random Access: Stacks do not allow for random access to elements, making them unsuitable for applications where elements need to be accessed in a specific order.

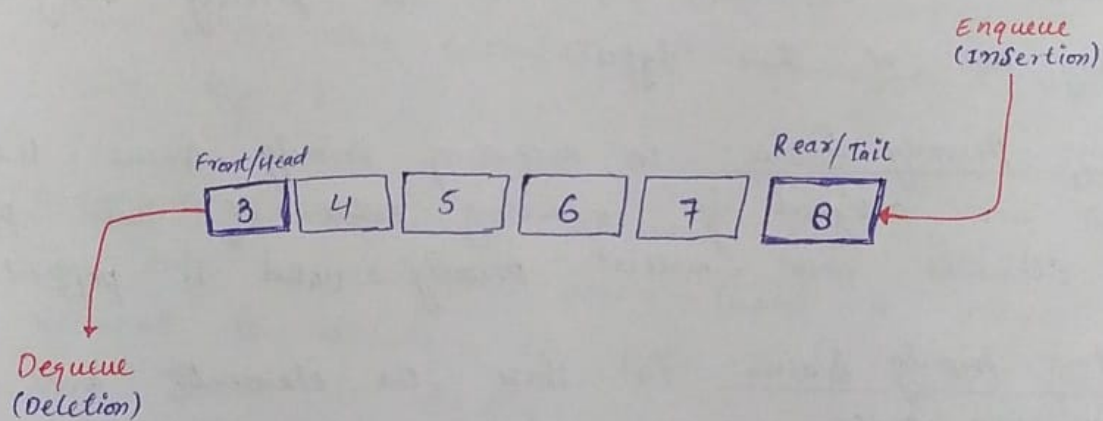
Limited Capacity: Stacks have a fixed capacity, which can be a limitation if the number of elements that need to be stored is unknown or highly variable.

QUEUE

Queue Data Structure is a linear Data Structure that is open at both ends and the operations are performed in First In First Out (FIFO) order. A Queue is a data structure in which whatever comes first will go out first.

Queue can also be defined as the list or collection in which the insertion is done from one end known as the rear end or the tail of the Queue, whereas the deletion is done from another end known as the front end or the Head of the Queue.

Representation:



TYPES:

Queue data structure can be classified into 4 types:

1. Simple Queue: Simple Queue simply follows FIFO structure. we can only insert the element at the back and remove the element from the front of the queue.

2. Double-Ended Queue (Deque):

In a double ended queue the insertion and Deletion operations, both can be performed from both ends. They are of two types:

① Input Restricted Queue: This is a simple Queue. In this type of Queue, the input can be taken from only one end but deletion can be done from any of the ends.

② Output Restricted Queue: This is also a simple Queue. In this type of queue, the input can be taken from both ends but deletion can be done from only one end.

• Circular Queue: This is special type of queue where the last position is connected back to the first position. Here also the operations are performed in FIFO order.

• Priority Queue: A priority queue is a special queue where the elements are accessed based on the priority assigned to them. They are of two types:

① Ascending Priority Queue: In Ascending Priority Queue, the elements are assigned in increasing order of their priority values. Elements with Smallest priority value is popped first.

② Descending Priority Queue: In these, the elements are arranged in decreasing order of their priority values. Elements with largest priority is popped first.

ARRAY REPRESENTATION OF QUEUE

To implement a queue using a Simple Array:

- Create an array of size n . and
- Take two variables `front` and `rear` which are initialized to -1 .

↳ Rear: is the index upto which the elements are stored in an array. we mainly add an item by incrementing it.

↳ front: is the index of the first element of the array. we mainly remove the element at this index in Dequeue operation.

↳ Enqueue: Addition of element to the queue. Adding an element will be performed after checking whether the Queue is full or not. If $\text{rear} == \text{MAX} - 1$, then it Queue is Full. But if array is not full then store the element at `queue[rear]`. and increment `rear` by 1.

↳ Dequeue: Removal of element from queue. An element can only be deleted when there is atleast an element to delete. If both `front` and `rear` are -1 then Queue is Empty.

↳ Display: Print all elements of Queue. If the Queue is non-Empty, traverse and print all the elements from the index `front` to `rear`.

CODE :

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5
```

```
struct queue {
    int front, rear;
    int que[MAX];
};
```

```
void addQueue (struct queue *, int);
```

```
void delQueue (struct queue *);
```

```
void display (struct queue *);
```

```
int main {
```

```
    struct queue q;
```

```
    q.front = q.rear = -1;
```

```
    int ch, val;
```

```
    do {
```

```
        printf("\n Choice : 1. AddQueue\n 2. delQueue\n 3. Exit\n 4. Display");
```

```
        printf("Enter your choice ");
```

```
        scanf("%d", &ch);
```

```
        switch (ch) {
```

```
            case 1:
```

```
                printf("Enter the value to Enqueue : ");
```

```
                scanf("%d", &val);
```

```
                addQueue(&q, val);
```

```
                break;
```

```
            case 2:
```

```
                delQueue(&q);
```

```
                break;
```

```
            case 3:
```

```
                printf("Exiting Program...\n");
```

```
                exit(0);
```


Case 4:

```
display(&q);
```

```
break;
```

```
default:
```

```
printf("Invalid Choice ! Try Again.\n");
```

```
}
```

```
} while(1);
```

```
return 0;
```

```
}
```

// Add Element to Queue.

```
void addQueue(struct queue *pq, int x){
```

```
if (pq->rear == MAX-1){
```

```
printf("Queue is Full\n");
```

```
return;
```

```
}
```

```
else if (pq->front == -1 && pq->rear == -1){
```

```
pq->front = pq->rear = 0;
```

```
pq->que[pq->rear] = x;
```

```
} else {
```

```
pq->rear++;
```

```
pq->que[pq->rear] = x;
```

```
}
```

```
printf("Added %d to the Queue.\n", x);
```

```
}
```

// Delete Element from Queue.

```
void delQueue(struct queue *pq){
```

```
if (pq->front == -1 && pq->rear == -1){
```

```
printf("Queue is Empty\n");
```

```
}
```

```

else if (pq->front == pq->rear) {
    printf("The deleted Element is : %d\n", pq->que[pq->front]);
    pq->front = pq->rear = -1;
} else {
    printf("The deleted Element is : %d\n", pq->que[pq->front]);
    pq->front++;
}
}

```

// Display the Queue.

```

void display(struct queue *pq) {
    if (pq->front == -1 && pq->rear == -1) {
        printf("Queue is Empty");
    } else {
        printf("Queue Elements are : ");
        for (int i = 0; i < pq->rear; i++)
        for (int i = pq->front; i <= pq->rear; i++) {
            printf("%d ", pq->que[i]);
        }
        printf("\n");
    }
}

```

Overall Time Complexity :

- addQueue $\rightarrow O(1)$
- delQueue $\rightarrow O(1)$
- display $\rightarrow O(n)$ (where n is the number of elements in the queue (bounded by max))

Circular Queue :

A circular Queue is similar to a linear queue as it is also based on FIFO principle except that the last position is connected to the first position in a circular queue that forms a circle. It is also known as RING BUFFER.

Operations :

- Enqueue
- Dequeue

Applications :

- Memory management : circular Queue provides efficient memory management as it places elements in a location which is used.
- CPU Scheduling : The OS also uses circular queue to insert the processes and then execute them.
- Traffic System : In computer controlled traffic system, circular queues are used to switch on the traffic lights one by one repeatedly as per the time set.

Enqueue operation :

1. check if the queue is full
→ If $(front == 0 \ \&\& \ rear == MAX-1) \ || \ (rear+1 == front)$
Point Queue is full and exit.
2. If Queue is empty, initialize front and rear = 0.
3. If rear is at the end of array and wrapping around is possible then set rear = 0.
4. for general case (rear inc. normally) → increment rear.
5. Insert value into Queue → Set $que[rear] = val$.

Dequeue operation:

1. If the Queue is empty return -1:
if (front == -1)
return -1;
2. Store the front element
x = que[front]
3. Check if this is the last element.
if front == rear:
reset front and rear to -1.
4. If front reaches the last index, wrap around to 0.
if front == max-1.
front = 0.
5. Otherwise increment front.
6. Return deleted element.

CODE:

```
#define MAX 5
struct queue {
    int front, rear;
    int que[MAX];
};
```

```
void enqueue(struct queue *, int);
int dequeue(struct queue *);
```

```
int main () {
    struct queue cq;
    cq.front = cq.rear = -1;
```

```
    enqueue(&cq, 10);
```

```
    enqueue(&cq, 20);
```

```
    enqueue(&cq, 30);
```

```
    int x = dequeue(&cq);
```

```
    if (x == -1) {
```

```
        printf("Queue is Empty...");
```

```
    } else {
```

```
        printf("Deleted element is: %d", x);
```

```
    }
```

```
    return 0;
```

```
}
```



```

void enqueue (struct queue *cq, int val) {
    if ((cq->front == 0 && cq->rear == MAX-1) || (cq->rear+1 == cq->front)) {
        printf("Queue is full...\n");
        return;
    }
    else if (cq->front == -1) {
        cq->front = cq->rear = 0;
    }
    else if (cq->rear == MAX-1 && cq->front != 0) {
        cq->rear = 0;
    }
    else {
        cq->rear++;
    }
    cq->que[cq->rear] = val;
}

```

```

int dequeue (struct queue *cq) {
    if (cq->front == -1) {
        return -1;
    }
    int x = cq->que[cq->front];
    cq->que[cq->front] = 0;
    if (cq->front == cq->rear) {
        cq->front = cq->rear = -1;
    }
    else if (cq->front == MAX-1) {
        cq->front = 0;
    }
    else {
        cq->front++;
    }
    return x;
}

```

APPLICATIONS OF QUEUES

- Multi Programming: Multi programming means when multiple programs are running in the main memory. It is essential to organize these multiple programs and these multiple programs are organized as queues.
- Network: In a network, a queue is used in devices such as router or a switch.
↳ Another application of a queue is a mail queue which is a directory that stores data and controls files for mail messages.
- Job Scheduling: A computer executes a series of tasks or jobs that are arranged sequentially. These jobs are assigned to the processor one at a time, managed through a queue.
- Shared Resources: Queues are used as waiting lists for a single shared resource.

Advantages:

- A large amount of data can be managed efficiently.
- Operations such as insertion and deletion can be performed with ease as it follows FIFO.
- Queues are fast in speed for data inter process communication.
- Queues can be used in the implementation of other ds.

Disadvantages:

Searching an element takes $O(N)$ time.
Maximum size of a queue must be defined prior in case of array implementation.

In a classic queue, a new element can only be inserted when existing elements are deleted from queue.

Linked List

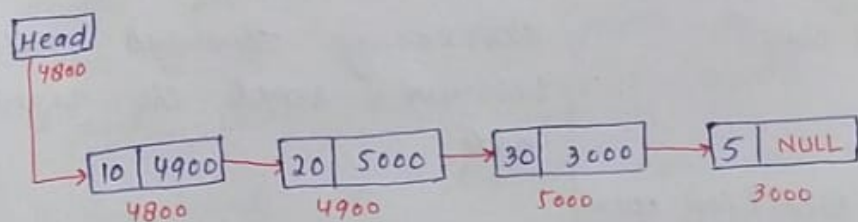
(1)

Linked List can be defined as a collection of objects called nodes that are stored randomly in the memory.

OR

Linked List is a collection of nodes in which one node is connected to another node.

- A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.
- The last node of the list contains pointer to the NULL.



- A linked list is a dynamic linear data structure whose memory size can be allocated or de-allocated at run time based on the operation insertion or deletion, this helps in using system memory efficiently.
- Linked lists can be used to implement various data structures like a stack, queue, graph, hash maps etc.

Uses of Linked List :

- The list is not required to be contiguously present in the memory. The node can reside anywhere in the memory and linked together to make a list. This achieves optimized utilization of space.

- ② - Empty node cannot be present in the Linked List.

→ Linked Lists vs Arrays :

Array

- Arrays are stored in contiguous location
- Fixed Size (Dynamically sized arrays also internally use fixed sized arrays)
- Only store elements no extra reference / pointer
- Elements can be accessed easily in $O(1)$ time
- Insertion and Deletion operation is slower than LL.

Linked List

- Linked lists are not stored in contiguous locations.
- Dynamic size.
- It stores both data and address of next node.
- Elements can be accessed by traversing through all nodes till we reach the required node
- Insertion and Deletion operations are faster than Arrays.

Declaration of Linked List :

In linked list, one is variable and second one is pointer variable. We can declare linked list by using user-defined data type called structure.

```
struct node {  
    int data;  
    struct node *next;  
};
```


Types of Linked List

(3)

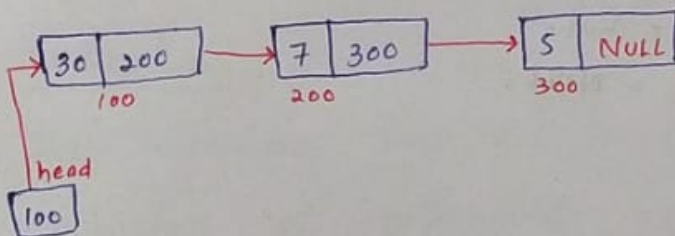
There are mainly three types of linked lists :

1. Singly Linked List.
2. Doubly Linked List.
3. Circular Linked List.

1. Singly Linked List :

It is a type of linked list where each node has two parts : data and next pointer.

- The data part stores the information and
- The next pointer points to the next node of the linked list.
- The next pointer of the last node stores NULL as it is the last node of the linked list and there is no next node.
- Traversal of items can be done in the forward direction only due to the linking of every node to its next node.



- The pointer that holds the address of the initial node is known as head pointer.

Node Definition:

```
struct Node {  
    int Data;  
    struct Node *next;  
}
```

→ Operations on Singly Linked List:

1. Node Creation:

```
#include <stdio.h>  
#include <malloc.h>
```

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

```
struct Node *Create (struct Node *head, int n) {
```

```
    int val;
```

```
    struct Node *r, *t;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (head == NULL) {
```

```
            head = (struct Node *) malloc (sizeof (struct Node));
```

```
            printf ("Enter value :- ");
```

```
            scanf ("%d", &val);
```

```
            head->data = val;
```

```
            head->next = NULL;
```

```
            r = head;
```

```
        }
```

```
        else {
```

```
            t = (struct Node *) malloc (sizeof (struct Node));
```

```
            printf ("Enter value ");
```

```
            scanf ("%d", &val);
```

```
            t->data = val;
```

```
            t->next = NULL;
```

```
            r->next = t
```

```
            r = t;
```

```
        }
```

```
    return head; }
```



```
void display(struct Node *head) {
```

(5)

```
while (head != NULL) {
```

```
    printf("%d -> ", head->data);
```

```
    head = head->next;
```

```
}
```

```
printf("NULL");
```

```
}
```

```
int main () {
```

```
    struct Node * head;
```

```
    int n;
```

```
    printf("Enter how many nodes to create");
```

```
    scanf("%d", &n);
```

```
    head = Create(head, n);
```

```
    printf("Nodes in linked list are :");
```

```
    display(head);
```

```
    return 0;
```

```
}
```

2. Insertion:

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted,

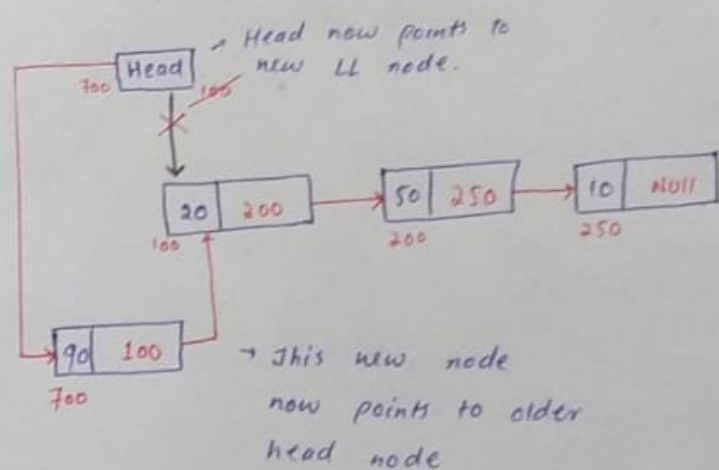
1. Insertion at beginning:

To insert a new node at the front, we create a new node and point its next reference to the current head of the linked list. Then we update the head to be this new node.

This operation is efficient because it only requires adjusting a few pointers. (6)

Algorithm:

1. Start
2. Create a new node.
3. Assign its data value.
4. Assign newly created node's next ptr to current head reference. So, it points to the previous start node of the linked list address.
5. Change the head reference to the new node's address.
6. End.



Code:

Assuming that Linked List Creation code is already present:

```
struct Node * addBeg(struct Node * p){
```

```
    int val;
```

```
    struct Node * t;
```

```
    t = (struct Node *) malloc(sizeof(struct Node));
```

```
    printf("Enter value : ");
```

```
    scanf("%d", &val);
```

```
    t->data = val;
```

```
    t->next = p;
```

```
    p = t;
```

```
    return p;
```

```
}
```

```
int main () {
```

```
    // call after LL creation
```

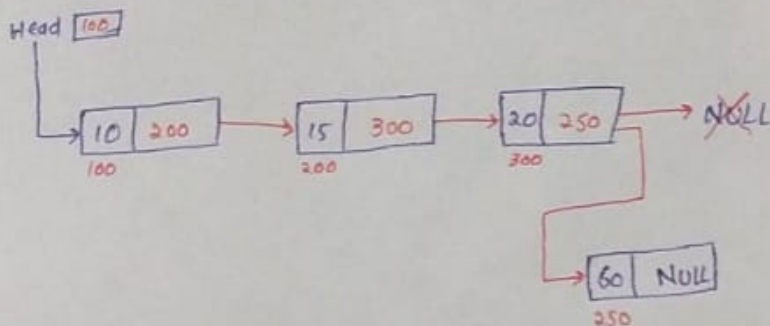
```
    head = addBeg(head);
```

```
    display(head);
```

```
}
```


2. Insert the Node at the End of Linked List

Inserting at the end involves traversing the entire list until we reach the last node. We then set the last node's next reference to point to the new node, making the new node the last element in the list.



Algorithm:

- Go to the last node of linked list
- Change the next pointer of last node from NULL to new node.
- Make the next pointer of new node as NULL to show the end of linked list.

Code:

Assuming that the linked list creation code and display code is already present.

```
void addEnd(struct Node *P) {
    struct Node *t;
    int val;
    while (P->next != NULL) {
        P = P->next;
    }
    t = (struct Node *) malloc(sizeof(struct Node));
    printf("\nEnter value: ");
    scanf("%d", &val);
    t->data = val;
    P->next = t;
    t->next = NULL;
}
```

```
int main() {
```

// Call after node creation -

```
printf("Adding node at end of list");
```

```
addEnd(head);
```

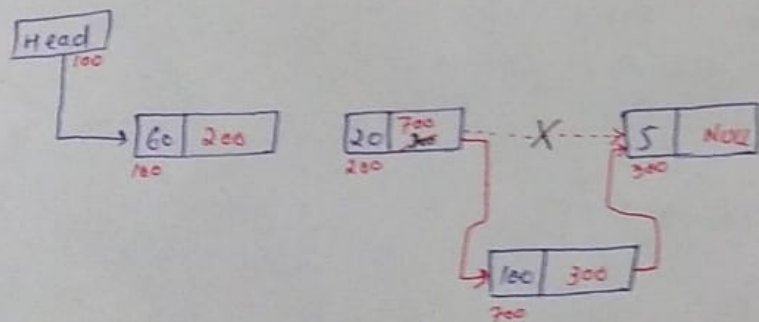
```
display(head);
```

```
return 0;
```

```
}
```

③ Insert a node At a Specific position in linked List :

To insert a new node at a specific position, we need to Traverse the list to position - 1. If the position is Valid, we adjust the pointers such that the next pointer of the new node points to the next of current node and next pointer of current node points to the new node.



Algorithm :

1. Check if position is valid, if not return with error message.
2. Create new node and get data from user.
3. If position/location is 1 :
 - Point new nodes next to head.
 - Return new node as head.
4. Traverse the linked list upto Location - 1 nodes.

9

- * If position exceeds list length, free new node and return.
- * Point the next pointer of the new node to the next of current node.
- Point the next pointer of current node to the new node.
- Return head pointer.

CODE :

```
struct Node * addAt (struct Node * head, int loc) {
    int val;
    struct Node * t, * curr;

    if (loc < 1) {
        printf ("Invalid location");
        return head;
    }

    t = (struct Node *) malloc (sizeof (struct Node));
    printf ("Enter value : ");
    scanf ("%d", &val);

    if (loc == 1) {
        t->next = head;
        return t;
    }

    curr = head;
    for (int i = 1; i < loc - 1; i++) {
        curr = curr->next;
        curr = curr->next;
        if (curr->next == NULL) {
            printf ("There are less nodes in list");
            free (t);
            return head;
        }
        curr = curr->next;
    }

    t->next = curr->next;
    curr->next = t;

    return head;
}
```

int main() {

// Linked list creation and other codes

...

printf("\n Enter location of insertion ");

scanf("%d", &loc);

head = addAt(head, loc);

printf("\n After insertion at %d location list is ", loc);

display(head);

return 0;

}

DELETION IN LINKED LIST

(11)

Deleting a node in a linked list is an important operation and can be done in 3 ways:

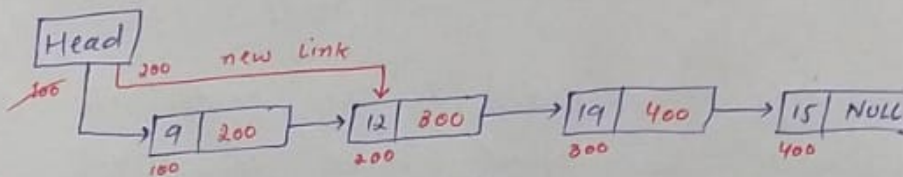
① Deletion at the beginning of linked list:-

It involves removing the first node of the linked list.

- To perform the deletion at the beginning of linked list, we need to change the head pointer to point to the second node.
- If the list is empty, there is nothing to delete.

Algorithm:

- Assign next node in a linked list as the new head.
- free previous head.



Code:

Assuming linked list is already created.

```
struct Node * delBeg (struct Node * p) {  
    struct Node * t;  
    if (p == NULL) {  
        printf("\n List is Empty ");  
        return NULL;  
    }  
    t = p;  
    p = p->next;  
    free(t);  
    return p;  
}
```

```
int main() {
```

// creation code here.

```
printf("Deleting Node at Beginning of List");  
head = delBeg(head);
```

```
Print  
display(head);
```

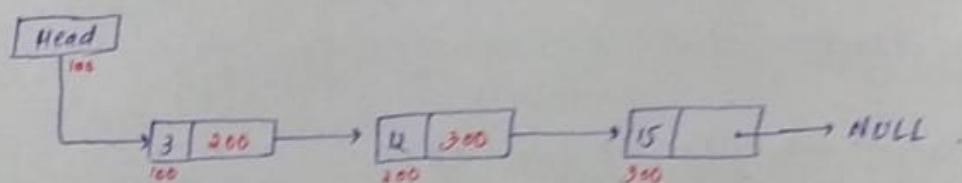
```
return 0;
```

```
}
```

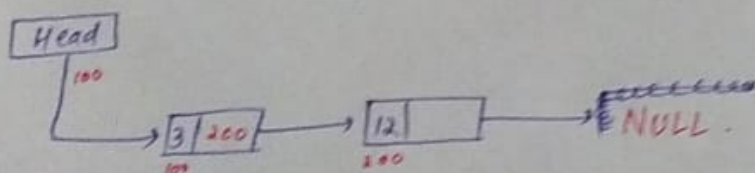
② Deletion at the End of List :

Deletion at end operation involves removing the last node of the linked list.

- To perform the deletion at the end of linked list, we need to traverse the list to find the second last node, then set its next pointer to NULL.
- If the list is empty then there is no node to delete or has only one node then point head to NULL.



↓ After Deletion.



Code:

Assuming Linked List is already created

13

```
void delEnd (struct Node *p){
```

```
    struct Node *t;
```

```
    if (p == NULL){
```

```
        printf("\n List is Empty");  
        return;
```

```
    }
```

```
while (p == NULL){
```

```
    while (p->next != NULL){
```

```
        t = p
```

```
        p = p->next;
```

```
    }
```

```
    t->next = NULL;
```

```
    free (p);
```

```
}
```

```
int main () {
```

```
    " LL creation code Here.
```

```
    :
```

```
    printf("\n Deleting Node at End of List");  
    delEnd(head);
```

```
    printf("\n After Deletion at End List is ");  
    display(head);
```

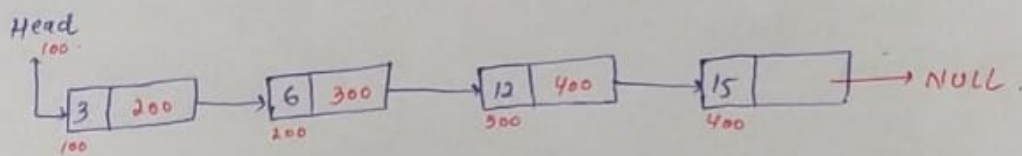
```
    return 0;
```

```
}
```

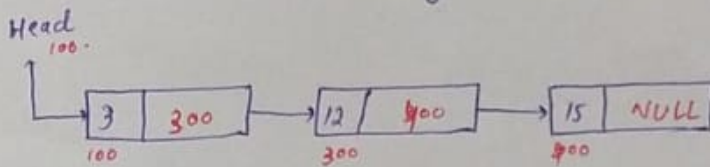
③ Deletion At specific Position of Linked List;

Deletion at a specified position in a Linked List involves removing a node from a specified index / position, which can be first, middle, or last node.

- To perform the deletion, if the position is 1, we update the head to point to next node and delete the current head. For other positions, we traverse the list to reach the node just before the specified position. If the target node exists, we adjust the next of this previous node to point to next of next nodes. which will result in skipping the target node.



Deleting node at Position = 2.



STEP BY STEP APPROACH :-

- If list is empty ($\text{head} == \text{NULL}$): return head pointer unchanged.
- If location is invalid ($\text{loc} < 1$): return head pointer unchanged.
- If deleting the first Node ($\text{loc} == 1$):
 - update $\text{head} = \text{head} \rightarrow \text{next}$;
 - return updated head.
- Traverse the list untill reaching the desired location:
 - Initialize t and r to keep track of current and previous nodes.
 - move t through the list untill the location is reached.

→ Check for Valid Location :

(15)

- If t becomes NULL, it means the location exceeds the list length. Return head.

→ If the node to delete is found :

- Set $r \rightarrow \text{next}$ to $t \rightarrow \text{next}$, effectively skipping over the node to be deleted.

- Free the node t .

- Return head.

Code:

```
struct Node * delAt (struct Node * head, int loc) {  
    struct Node * r, * t;  
  
    // Check for empty list.  
    if (head == NULL) {  
        printf("\n List is Empty");  
        return head;  
    }  
  
    if (loc < 1) {  
        printf("Invalid location");  
        return head;  
    }  
  
    if (loc == 1) {  
        t = head;  
        head = head → next;  
        free(t);  
        return head;  
    }  
  
    t = head;  
    for (int i = 0; i < loc; i++) {  
        r = t;  
        t = t → next;  
        if (t == NULL) {  
            printf("\n location Exceeds list length");  
            return head;  
        }  
    }
```

// Delete Node

(16)

$s \rightarrow \text{next} = t \rightarrow \text{next};$

$\text{free}(t);$

$\text{return head};$

}

$\text{int main}() \{$

// linked list creation code here.

:

$\text{printf}(\text{"Enter location of Deletion - "});$

$\text{scanf}(\text{"\%d"}, \&\text{loc});$

~~$\text{head} = \text{delAt}(\text{head}, \text{loc});$~~

$\text{head} = \text{delAt}(\text{head}, \text{loc});$

$\text{printf}(\text{"In After Deletion at \%d location, list is\n"}, \text{loc});$

$\text{display}(\text{head});$



Time Complexity Analysis of Linked List.

Operation	Time
Random Access	$O(n)$
Insertion and Deletion at Beginning	$O(1)$
Insertion and Deletion at end	$O(n)$
Insertion and Deletion at random position	$O(n)$

SEARCH AN ELEMENT IN A LINKED LIST

(17)

- Given a linked list and a key, the task is to check if key is present in the linked list or not?

① Iterative Approach: $O(N)$ Time and $O(1)$ Space

The idea is to traverse all the nodes of the linked list, starting from head. while traversing, if we find a node whose value is equal to key then Print 'Yes' otherwise 'No'.

STEPS:

- Initialize a node pointer, $curr = head$;
- Do following while $curr$ is not NULL.
 - If the $curr$ value (i.e.: $curr \rightarrow data$) is equal to the key being searched, Print 'YES' and return.
 - Otherwise, move to the next node ($curr = curr \rightarrow next$);
- If the key is not found, print 'NO'.

CODE:

```
void searchList (struct Node *head, int key) {  
    struct Node *curr;  
    curr = head;  
    int index = 1;  
    while (curr != NULL) {  
        if (curr->data == key) {  
            printf("In YES, key is present at %d location :", index);  
            return;  
        }  
        curr = curr->next;  
        index++;  
    }  
    printf("In No, key is not Present ");  
}
```

// Now call this function in main.

② Recursive Approach : $O(N)$ Time and $O(N)$ Space

The idea is to Recursively traverse all the nodes starting from the head of linked list. For any node, if the value is equal to key, then return true. Otherwise recursively search for next node. If at any point the head reaches NULL, it means that we have reached the end of linked list. So return false.

STEPS :

- If the head is NULL return false.
- If the head's data is same as key, return true.
- Else recursively search for next node.

CODE: `#include <stdbool.h>`

```
bool searchkey (struct Node * head, int key) {
    if (head == NULL) {
        return 0;
    }
    if (head->data == key) {
        return 1;
    }
    return searchkey (head->next, key);
}
```

```
int main () {
```

```
    // creation code.
```

```
    :
```

```
    // function call (Take key from user).
```

```
    if (searchkey (head, key) {
```

```
        printf("In YES, Key is Present");
```

```
    } else {
```

```
        printf("In NO, Key is not Present");
```

```
    }
```


REVERSE A LINKED LIST (iterative method - $O(n)$ Time and $O(1)$ Space) (19)

Given a linked list, the task is to reverse the linked list by changing the links b/w nodes.

→ The idea is to Reverse the links of all nodes using three pointers:

- prev: Pointer to keep track of previous node.
- curr: Pointer to keep track of current node.
- next: Pointer to keep track of next node.

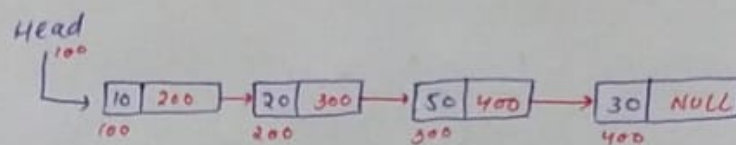
→ Starting from the first node, initialize curr with the head of linked list and next with the next node of curr.

Update the next pointer of curr with prev.

Finally, move the three pointer by updating prev with curr and curr with next.

memory map:

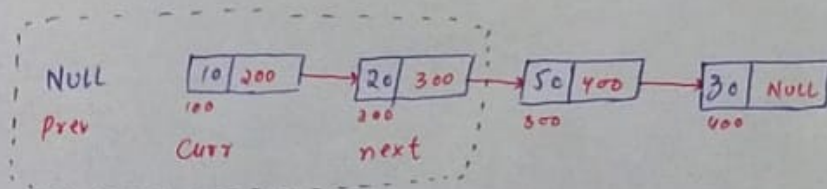
Step 1: Input linked list with 4 nodes.

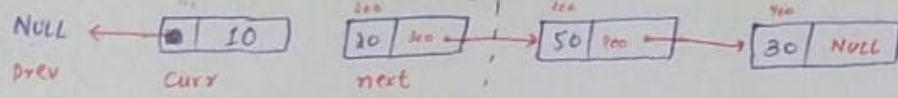


Step 2: Initialize prev pointer = NULL

Store next = curr → next.

Update curr → next = prev.



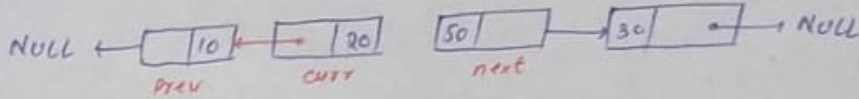
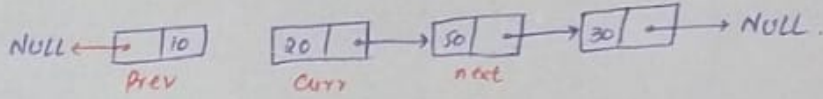


20

STEP 3: update : $prev = curr$ and $curr = next$.

Store next node 3 as next.

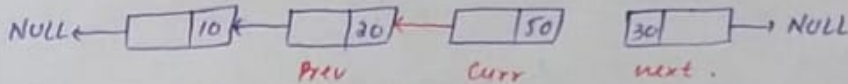
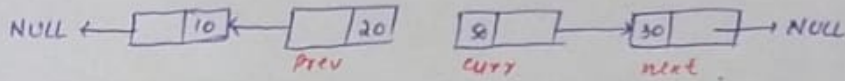
Update next pointer of curr (node 2) to prev (node 1).



STEP 4: update $\Rightarrow prev = curr$ and $curr = next$.

Store next (node 4) as next.

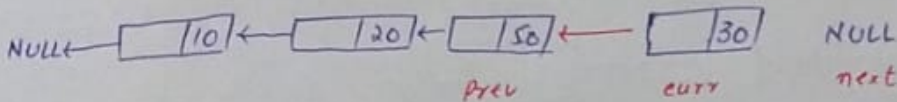
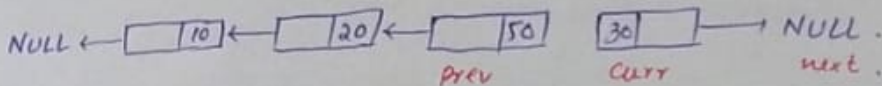
Update next pointer of curr (node 3) to prev (node 2).



STEP 5: update $\Rightarrow prev = curr$ and $curr = next$.

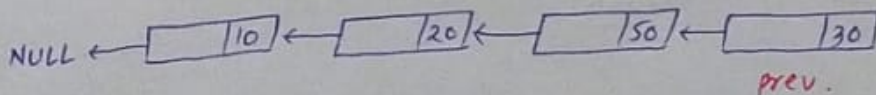
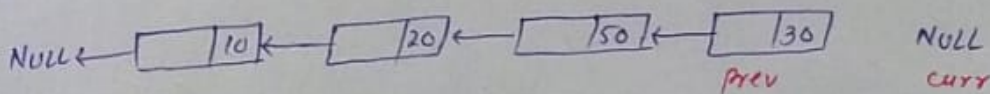
next pointer points to NULL.

Update next pointer of curr (node 4) to prev (node 3).



STEP 6: update $prev = curr$ and $curr = next$.

Finally, curr becomes NULL and prev stores the head of reversed linked list.



CODE :

31

```
struct Node * reverse (struct Node * head) {
```

```
    struct Node * prev, * curr, * next;
```

```
    curr = head;
```

```
    prev = NULL.
```

```
    while (curr != NULL) {
```

```
        next = curr → next;
```

```
        curr → next = prev;
```

```
        prev = curr;
```

```
        curr = next;
```

```
    }
```

```
    return prev
```

```
}
```

```
int main() {
```

```
    // LL. code ...
```

```
    :
```

```
    head = reverse(head);
```

```
    printf("In After Reverse List is \n");
```

```
    display(head);
```

```
}
```